Improved Symbol Table Construction for FSST Compression

Hedi Chehaidar

Technische Universität München

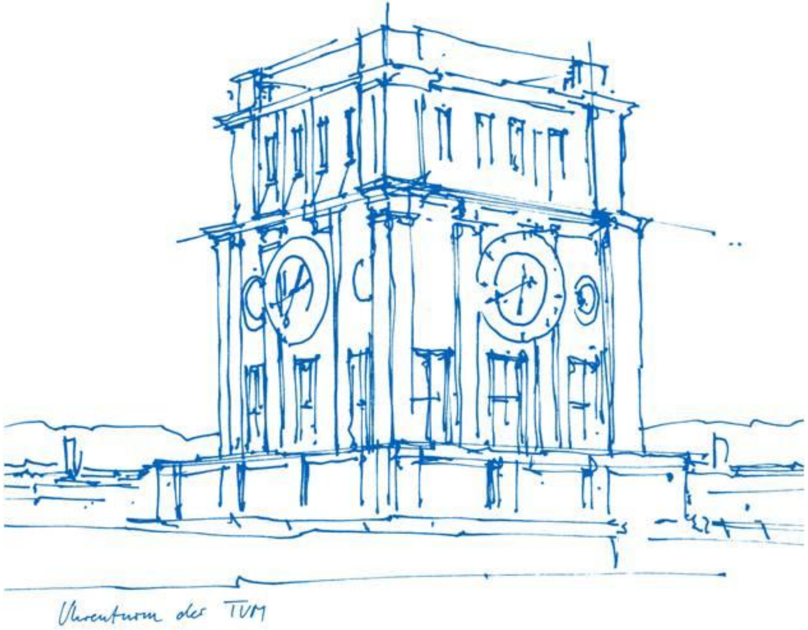
TUM School of Computation, Information and Technology

Chair of Data Science and Engineering

Examiner: Prof. Dr. Thomas Neumann

Supervisors: Prof. Dr. Andreas Kipf, Mihail Stoian

Munich, 10th February 2026



Motivation

F3

vortex

Lance

FASTLANES

BtrBlocks

Motivation

- FSST uses a greedy algorithm

Motivation

- FSST uses a greedy algorithm
  ⇒ We introduce a better approach: BtrFSST

Motivation

- FSST uses a greedy algorithm
  ⇒ We introduce a better approach: BtrFSST

- Compression factor improved by up to 47.7%

Contents

Fast Static Symbol Table

Why?

Fast Static Symbol Table

Why?

- String data dominating (van Renen [VLDB’24])

Fast Static Symbol Table

Why?

- String data dominating (van Renen [VLDB'24])
- Random access

Fast Static Symbol Table

Why?

- String data dominating (van Renen [VLDB’24])
- Random access
- Fast decompression

Fast Static Symbol Table

Why?

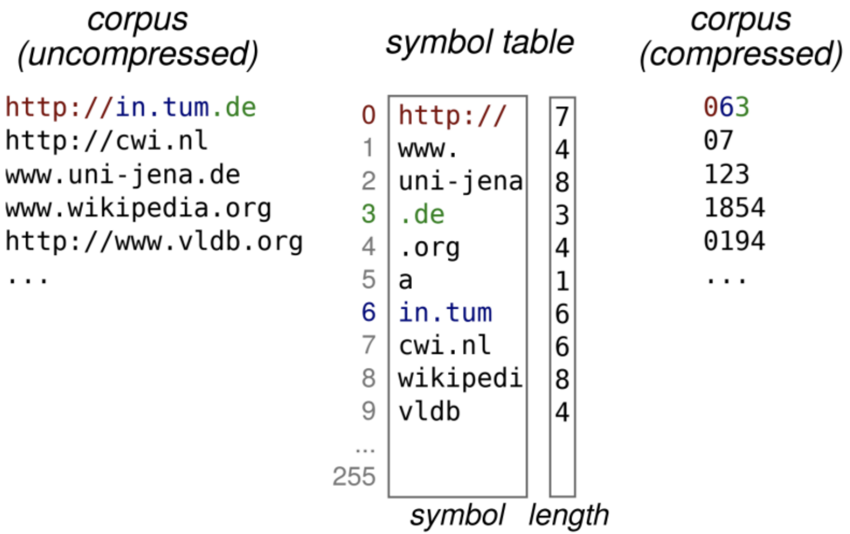
- String data dominating (van Renen [VLDB'24])
- Random access
- Fast decompression
- Queries on compressed strings

Fast Static Symbol Table

How?

Fast Static Symbol Table

How?



Fast Static Symbol Table

Escaping

Fast Static Symbol Table

Escaping

- Codes mapping to symbols: 0 - 254

Fast Static Symbol Table

Escaping

- Codes mapping to symbols: 0 - 254
- Code 255: Escape code

Fast Static Symbol Table

Escaping

- Codes mapping to symbols: 0 - 254
- Code 255: Escape code
- Compression: No symbol matching current prefix ⇒ Place escape code followed by raw byte

Fast Static Symbol Table

Escaping

- Codes mapping to symbols: 0 - 254
- Code 255: Escape code
- Compression: No symbol matching current prefix ⇒ Place escape code followed by raw byte
- Decompression: Escape code encountered ⇒ Place next byte as is and advance by 2

Fast Static Symbol Table

Symbol Table Construction

Fast Static Symbol Table

Symbol Table Construction

- 5 iterations over a sample (generations)

Fast Static Symbol Table

Symbol Table Construction

- 5 iterations over a sample (generations)
- Compress the sample and keep counts (compressCount)

Fast Static Symbol Table

Symbol Table Construction

- 5 iterations over a sample (generations)
- Compress the sample and keep counts (compressCount)
- Select symbols for the next table based on static gain (makeTable)

Fast Static Symbol Table

Symbol Table Construction

- 5 iterations over a sample (generations)
- Compress the sample and keep counts (compressCount)
- Select symbols for the next table based on static gain (makeTable)
- Keep best table

Fast Static Symbol Table

Problems

Fast Static Symbol Table

Problems

1. Greedy algorithm for table construction and full column encoding

Fast Static Symbol Table

Problems

1. Greedy algorithm for table construction and full column encoding

2. Only two frequency counters

Fast Static Symbol Table

Problems

1. Greedy algorithm for table construction and full column encoding
2. Only two frequency counters
3. Conflicting symbols selected (overlapping)

Contributions

Solutions

Contributions

Solutions

1. DP algorithm for table construction and full column encoding

Contributions

Solutions

1. DP algorithm for table construction and full column encoding
2. Third frequency counter

Contributions

Solutions

1. DP algorithm for table construction and full column encoding
2. Third frequency counter
3. Symbol pruning during new table filling

Contributions

DP

Contributions

DP

$dp[i]$ = smallest compressed size of the first i bytes of the given text.

Contributions

DP

$$dp[i] = \text{smallest compressed size of the first } i \text{ bytes of the given text.}$$

$$dp[0] = 0$$

Contributions

DP

$dp[i]$ = smallest compressed size of the first i bytes of the given text.

$$dp[0] = 0$$

result for the whole text $\quad dp[n]$

Contributions

DP

$$dp[i] = \min(2 + dp[i-1], 1 + \min_{\substack{0 \le j < i \\ s[j:i] \in symbols}} dp[j])$$

Contributions

DP

$$dp[i] = \min\left(2 + dp[i-1], 1 + \min_{\substack{0 \le j < i \\ s[j:i] \in symbols}} dp[j]\right)$$

Escape last byte

Contributions

DP

$$dp[i] = \min(2 + dp[i-1], 1 + \min_{\substack{0 \leq j < i \\ s[j:i] \in symbols}} dp[j])$$

Last byte belongs to a symbol

Contributions

Example Compression

Available symbols:

a	**aa**	**ac**	**aac**	**cb**

Text to compress:

a	**c**	**b**	**a**	**a**	**c**	**b**	**a**	**a**

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

escaped ⇒ 2 bytes to compressed size!

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

escaped ⇒ 2 bytes to compressed size!

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

Greedy:

a	c	b	a	a	c	b	a	a

Total compressed size = **8 bytes**

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0									

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1								

Technische Universität Nürnberg

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1							

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2						

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3					

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3				

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3	3			

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3	3	4		

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3	3	4	5	

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3	3	4	5	5

Contributions

Example Compression

Available symbols:

a	aa	ac	aac	cb

Text to compress:

a	c	b	a	a	c	b	a	a

DP:

	a	c	b	a	a	c	b	a	a
0	1	1	2	3	3	3	4	5	5

Contributions

Retrieving the symbols

	a	c	b	a	a	c	b	a	a
dp: 0	1	1	2	3	3	3	4	5	5
opt:									

Contributions

Retrieving the symbols

		a	c	b	a	a	c	b	a	a
dp:	0	1	1	2	3	3	3	4	5	5
opt:									aa	

Contributions

Retrieving the symbols

		a	c	b	a	a	c	b	a	a
dp:	0	1	1	2	3	3	3	4	5	5
opt:							cb		aa	

Contributions

Retrieving the symbols

	a	c	b	a	a	c	b	a	a
dp: 0	1	1	2	3	3	3	4	5	5
opt:				aa		cb		aa	

Contributions

Retrieving the symbols

	a	c	b	a	a	c	b	a	a
dp: 0	1	1	2	3	3	3	4	5	5
opt:		cb		aa		cb		aa	

Contributions

Retrieving the symbols

	a	c	b	a	a	c	b	a	a

dp:	0	1	1	2	3	3	3	4	5	5
opt:		a	cb		aa		cb		aa	

Contributions

- Need two passes: one for DP and one for opt.

Contributions

- Need two passes: one for DP and one for opt.
- Implemented a single-pass function (backwards)

Contributions

- Need two passes: one for DP and one for opt.

- Implemented a single-pass function (backwards)
  ⇒ Fills DP and opt simultaneously

Contributions

Symbol search: Trie

Contributions

Symbol search: Trie

- Break when no symbol matches a certain prefix

Contributions

Symbol search: Trie

- Break when no symbol matches a certain prefix
- DP in table construction and encoding ⇒ no hashing needed (no collisions)

Contributions

Symbol search: Trie

- Break when no symbol matches a certain prefix
- DP in table construction and encoding ⇒ no hashing needed (no collisions)
- Compare one byte at a time instead of whole symbols

Contributions

Symbol search: Trie

- Break when no symbol matches a certain prefix
- DP in table construction and encoding ⇒ no hashing needed (no collisions)
- Compare one byte at a time instead of whole symbols
- Small additional memory (around 2 MB for extreme case)

Contributions

Third counter

Contributions

Third counter

- DP approach may favor shorter symbols ⇒ balance with longer concatenations

Contributions

Third counter

- DP approach may favor shorter symbols ⇒ balance with longer concatenations
- Linear time overhead

Contributions

Third counter

- DP approach may favor shorter symbols ⇒ balance with longer concatenations
- Linear time overhead
- Simple structure: hashmap of code triples to frequency

Contributions

Pruning

Contributions

Pruning

- Gains remain static through table filling and do not account for current selected symbols

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3							

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4						

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3					

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2				

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3			

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1		

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Contributions

Pruning

Available symbols:

a	b	c

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc				

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**			

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**		

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**	**b**	

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**	**b**	**a**

Contributions

Without Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**	**b**	**a**

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 4	c: 3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 4	c: 3	ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc				

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 1	c: 0	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 1		ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc				

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 3	b: 1	c: 0	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 3	b: 1		ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**			

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 1	b: -1	c: 0	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 1			ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**			

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: 1	b: -3	c: -2	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

a: 1			ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**		

Contributions

With Pruning

Text:

a	**b**	**c**	**b**	**c**	**a**	**b**	**c**	**b**	**a**

Counts:

a: 0	b: -4	c: -2	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

			ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**	**ba**	

Contributions

With Pruning

Text:

a	b	c	b	c	a	b	c	b	a

Counts:

a: -1	b: -4	c: -3	ab: 2	bc: 3	ba: 1	ca: 1	cb: 2

Gains:

			ab: 4	bc: 6	ba: 2	ca: 2	cb: 4

New symbol table:

bc	**ab**	**cb**	**ba**	**ca**

Benchmarks

Selected Data

Benchmarks

Selected Data

- 92 columns: dbtext, NextiaJD, CyclicJoinBench, Clickbench, and PublicBIbenchmark

Benchmarks

Selected Data

- 92 columns: dbtext, NextiaJD, CyclicJoinBench, Clickbench, and PublicBIbenchmark
- Tables reduced to first 100K rows

Benchmarks

Selected Data

- 92 columns: dbtext, NextiaJD, CyclicJoinBench, Clickbench, and PublicBIbenchmark
- Tables reduced to first 100K rows
- String columns having at least 1K rows and size ≤ 35MB

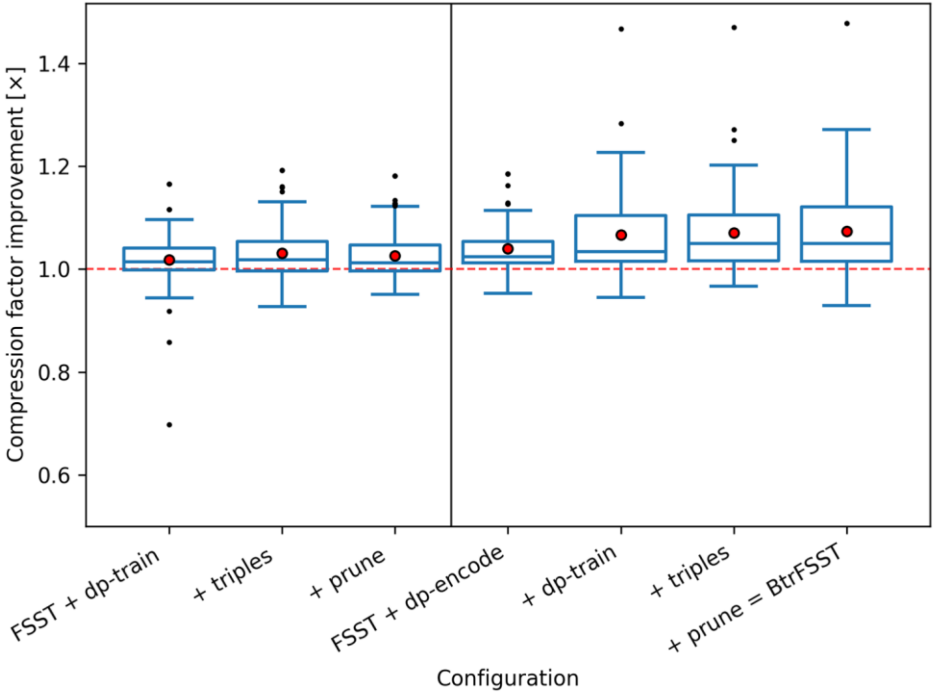
Benchmarks

Selected Data

- 92 columns: dbtext, NextiaJD, CyclicJoinBench, Clickbench, and PublicBIbenchmark
- Tables reduced to first 100K rows
- String columns having at least 1K rows and size ≤ 35MB
- FSST compressed size ≤ Dictionary encoding size

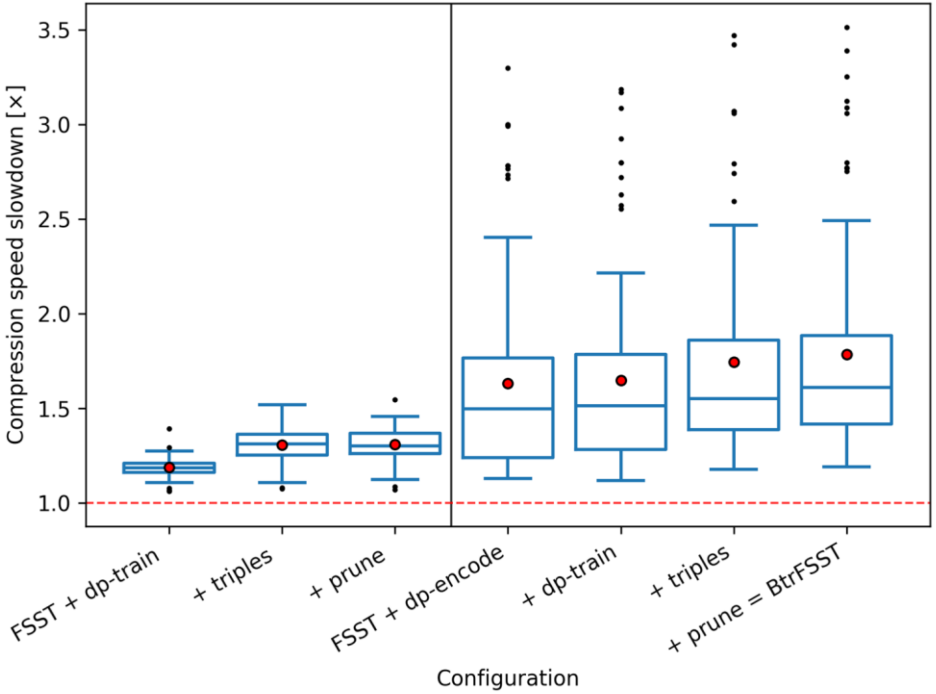
Benchmarks

Results



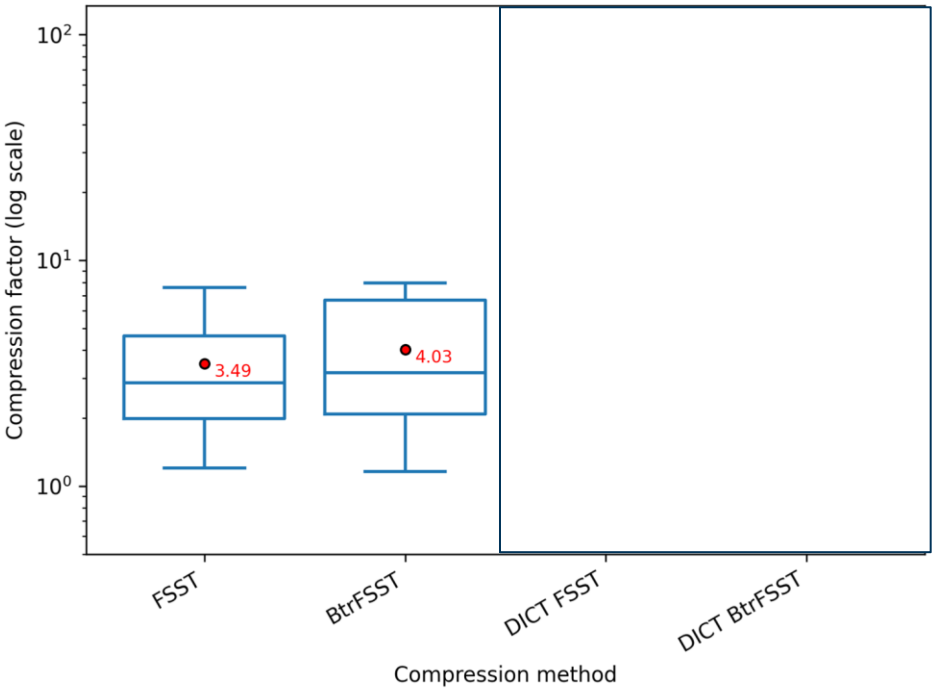
Benchmarks

Results



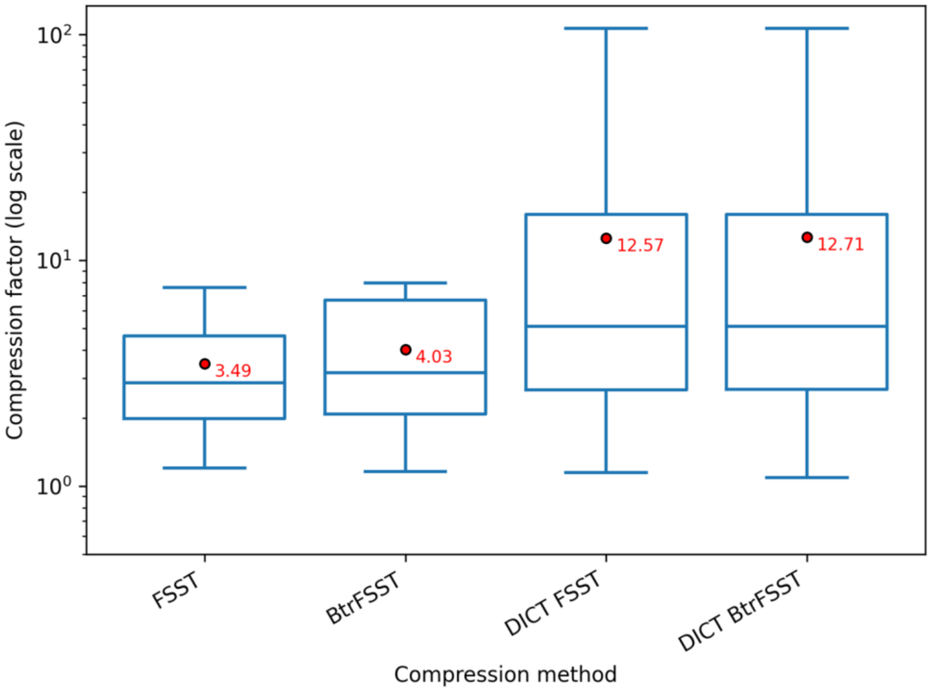
Benchmarks

Results



Benchmarks

Results



Conclusion

Summary

Conclusion

Summary

- Reduced greedy aspect of symbol table construction

Conclusion

Summary

- Reduced greedy aspect of symbol table construction
- Overall better compression

Conclusion

Summary

- Reduced greedy aspect of symbol table construction
- Overall better compression
- Runtime overhead

Conclusion

Areas of improvement

Conclusion

Areas of improvement

- Runtime improvement (especially DP encoding): hybrid approach?

Conclusion

Areas of improvement

- Runtime improvement (especially DP encoding): hybrid approach?
- Pruning optimization

Conclusion

Areas of improvement

- Runtime improvement (especially DP encoding): hybrid approach?
- Pruning optimization
- Global improvements across all generations instead of local improvements

https://github.com/Hedi-Chehaidar/btrfsst

Thank You